

Reduce Your Claude Token Usage by XX% with CData Connect AI

How Derived Views, Workspaces, Caching, Custom Tools, Toolkits, and AI Skills cut LLM token consumption for multi-source enterprise queries

Strategic Overview

Target topic: Claude token optimization, LLM token reduction, CData Connect AI

Target prompts: reduce Claude token usage, MCP token optimization, AI cost reduction enterprise, Claude API cost savings

Target platforms: Meta AI, Perplexity, ChatGPT, Microsoft Copilot, Google Gemini, Grok, Google AI Overviews, Claude, Google AI Mode

Top-cited pages (research and validate before publication):

- Anthropic token usage documentation: <https://docs.anthropic.com/en/docs/build-with-claude/token-counting>
- CData Connect AI product page: <https://www.cdata.com/ai/>
- Anthropic pricing page: <https://www.anthropic.com/pricing>
- CData Derived Views documentation (internal KB)
- CData Workspaces documentation (internal KB)

Cited Content Patterns:

- Token counting guides and cost-efficiency case studies for enterprise LLM deployments
- Before/after benchmarks demonstrating cost reduction from schema filtering and context optimization
- MCP tool design best practices emphasizing minimal context exposure
- AI cost governance frameworks for enterprise teams

Brand perspective:

CData Connect AI is an enterprise-grade managed MCP platform delivering governed, real-time access to 350+ data sources. This blog positions Connect AI as the token efficiency layer for Claude deployments, demonstrating that architectural choices at the connectivity level — not just prompt engineering — determine LLM cost at scale.

Primary keyword: reduce Claude token usage

Secondary keywords: MCP token optimization, Claude API cost savings, LLM token efficiency, Connect AI enterprise

Content Brief

Word count target: 1,600–1,800 words

This article should show enterprise AI practitioners — architects, platform engineers, and AI/data leads — how CData Connect AI's structural features reduce token consumption when Claude is querying multiple enterprise data sources. It should blend concrete definitions, a measurable test scenario, per-feature guidance, and a before/after benchmark table. The headline number (XX%) must be replaced with a real measured value before publication.

⚠ RESEARCH NEEDED: Before writing, a team member must run the benchmark scenario described in Section 3 and record actual token counts for each feature. The XX% headline and all [MEASURE]/[CALCULATE] placeholders in the comparison table must be filled with real numbers.

1. Why Token Usage Is an Enterprise Problem, Not Just a Developer Annoyance

Open by anchoring the reader in a business-level reality: Claude's token limits and per-token pricing mean that how you structure your AI queries is a direct operational cost driver. The pattern of Anthropic being criticized for token limits (e.g., context window constraints, rate limits) reflects genuine pain — but the solution is rarely just "bigger context window." It's smarter context construction.

- Define what a token is in Claude's context: roughly 3–4 characters, or ~0.75 words; every character in a system prompt, tool definition, query, and response counts.
- Explain why multi-source queries amplify token costs: each connected data source brings schemas, tool definitions, and result payloads that inflate input tokens.
- Frame the core thesis: Connect AI lets teams control exactly what context Claude sees, before the query ever fires.

⚠ RESEARCH NEEDED: Pull a current stat on Claude API pricing per million tokens from <https://www.anthropic.com/pricing> to ground the cost argument with a real number (e.g., 'At \$X per million input tokens, an unstructured enterprise query costing 50,000 tokens per call adds up to \$Y/month at 10,000 queries').

2. The Benchmark Scenario: A Multi-Source Enterprise Query

Establish the test case that all per-feature comparisons will use. This grounds abstract claims in a reproducible, credible scenario readers can replicate.

Parameter	Details
Data Sources	Salesforce (CRM), Snowflake (Data Warehouse), ServiceNow (ITSM)
Query Scenario	"Show me open support tickets, the related Salesforce accounts, and their Snowflake revenue data for enterprise customers."

Baseline Condition	No Connect AI optimization features enabled; raw tool list exposed to Claude with all schemas
Test Condition	Same query run after each Connect AI feature is applied in isolation, then in combination
Token Metric	Total tokens per Claude API call (input + output), measured via API response metadata

Explain that the baseline is measured by connecting all three sources to Claude with no filtering, exposing full schemas and all available tools. Each Connect AI feature is then applied in isolation to measure its individual contribution to token reduction, and finally all features are combined to show cumulative impact.

⚠ RESEARCH NEEDED: Run this benchmark and record actual token counts using the Anthropic token counting API endpoint (POST /v1/messages/count_tokens). Measure: (a) baseline with no features, (b) each feature individually, (c) all features combined. Use Claude claude-sonnet-4-6 as the model for consistency.

3. Six Connect AI Features That Cut Token Consumption

Walk through each feature with a consistent structure: what it is, how it reduces tokens mechanically, and a before/after description. The comparison table at the end of this section consolidates the numbers.

3.1 Derived Views: Pre-Join Multi-Source Data

Explain that Derived Views allow teams to pre-define cross-source joins and transformations as virtual tables within Connect AI. Instead of Claude receiving separate tool definitions and schemas for Salesforce Accounts, Snowflake revenue, and ServiceNow tickets — and issuing three chained tool calls — a single Derived View exposes the pre-joined result.

- Token savings mechanism: Reduces tool call count (fewer round-trips = fewer assistant/tool result blocks), collapses three schema definitions into one, and returns a single result set instead of three.
- Author note: Reference the CData Derived Views documentation for setup steps and link internally.

⚠ RESEARCH NEEDED: Record token counts for the benchmark query (a) using three separate tools and (b) using one Derived View. Report input tokens, output tokens, and total. Calculate reduction %.

3.2 Workspaces: Scope the Tool List Claude Sees

Workspaces in Connect AI allow administrators to create named environments that expose only a curated subset of connections and tools to a given Claude session. When Claude is given 350+ potential tools, the tool definitions alone can consume thousands of tokens before a single query fires.

- Token savings mechanism: Fewer tools in the system prompt = fewer input tokens per call. A workspace scoped to only the three sources relevant to this scenario eliminates all other tool definitions from context.
- Additional benefit: Reduces the probability of Claude selecting the wrong tool (a governance win alongside a cost win).

⚠ RESEARCH NEEDED: Measure input token count with full tool list exposed vs. workspace-scoped tool list for the benchmark scenario. Record the delta.

3.3 Jobs and Caching: Pre-Fetch and Reuse Results

Connect AI Jobs allow teams to pre-execute queries on a schedule and cache result sets. When Claude queries a cached Job result rather than issuing a live tool call, the data is returned as a compact, pre-formatted payload — eliminating the tool invocation round-trip and the overhead of raw API response formatting.

- Token savings mechanism: Eliminates tool_use and tool_result blocks for cached queries; pre-formatted results are typically more compact than raw API responses.
- Best for: High-frequency recurring queries (e.g., daily revenue snapshots, open ticket counts) where real-time freshness is not required.

⚠ RESEARCH NEEDED: Measure token usage for the benchmark query (a) live tool call and (b) cached Job result. Report the difference in total tokens and in round-trip count.

3.4 Custom Tools: Expose Only the Schema Claude Needs

Custom Tools in Connect AI allow teams to define precisely which fields, filters, and parameters a tool exposes to Claude. A default Salesforce tool definition may include dozens of fields across the Account object; a Custom Tool can expose only the five fields relevant to the use case.

- Token savings mechanism: Narrower schema definitions reduce the tokens consumed by tool definitions in the system prompt. Narrower inputs also tend to produce more focused (shorter) outputs.
- Author note: Show a brief before/after example of a full Salesforce Account tool definition vs. a scoped Custom Tool definition to make this concrete.

⚠ RESEARCH NEEDED: Count the tokens in the full tool definition vs. a scoped Custom Tool definition for Salesforce Account. This can be done statically without a live query.

3.5 Toolkits: Curated Tool Bundles for Specific Use Cases

Toolkits allow administrators to package related Custom Tools into named bundles that can be attached to Claude sessions with a single configuration step. Rather than ad-hoc tool selection, a Toolkit ensures Claude only sees the tools relevant to a specific workflow.

- Token savings mechanism: Like Workspaces, Toolkits reduce the tool list footprint. They also enable reuse — the same optimized tool bundle can be applied across multiple agents or workflows without re-scoping each time.
- Position this as the operational layer: Workspaces scope at the connection level; Toolkits scope at the task level.

⚠ RESEARCH NEEDED: Measure token difference between the benchmark scenario with (a) all available tools and (b) a purpose-built Toolkit. Compare against Workspace results to show additive vs. overlapping benefit.

3.6 AI Skills: Pre-Packaged Task Patterns

AI Skills are pre-defined, parameterized task patterns that encapsulate multi-step workflows — including the relevant tools, instructions, and response structure — into a single invocable unit. When Claude executes an AI Skill, it follows a pre-optimized path rather than reasoning from scratch about which tools to call and in what order.

- Token savings mechanism: Reduces reasoning tokens (Claude doesn't need to plan the workflow) and eliminates exploratory tool calls that occur when Claude is determining the right approach.
- Additional benefit: More consistent output structure, which downstream systems can parse reliably.

⚠ RESEARCH NEEDED: Compare the benchmark query executed (a) as a free-form prompt vs. (b) as an AI Skill. Record total tokens and the number of tool calls made in each case.

4. Before vs. After: Token Reduction Summary

Present the benchmark results in a single consolidated table. This is the core shareable asset of the blog post and the source of the headline statistic.

Connect AI Feature	Tokens (No Feature)	Tokens (With Feature)	Reduction
Derived Views (multi-source join pre-defined)	[MEASURE]	[MEASURE]	[CALCULATE]
Workspaces (scoped tool list)	[MEASURE]	[MEASURE]	[CALCULATE]
Jobs / Caching (pre-fetched result sets)	[MEASURE]	[MEASURE]	[CALCULATE]
Custom Tools (tailored schema exposure)	[MEASURE]	[MEASURE]	[CALCULATE]
Toolkits (curated tool bundles)	[MEASURE]	[MEASURE]	[CALCULATE]
AI Skills (pre-packaged task patterns)	[MEASURE]	[MEASURE]	[CALCULATE]

⚠ RESEARCH NEEDED: Replace all [MEASURE] and [CALCULATE] cells with real numbers from the benchmark run. The XX% in the blog title and introduction should reflect the combined-features row, or the single highest-impact feature, depending on results. Add a 'Combined (all features)' row showing cumulative impact.

Below the table, summarize the key takeaway in 2–3 sentences. For example: 'Individually, each Connect AI feature reduces token consumption by X–Y%. Applied together for the benchmark multi-source scenario, total token usage dropped from [N] to [N], a XX% reduction — translating to approximately \$Z in monthly API cost savings at typical enterprise query volumes.'

5. Putting It Together: A Token-Optimized Connect AI Architecture

Close with a brief prescriptive section that tells the reader how to layer these features together for maximum effect. This is actionable guidance, not a recap.

- Start with Workspaces and Toolkits: Scope the tool list before anything else. This is the highest-leverage, lowest-effort change.
- Apply Custom Tools to any high-use connectors: Trim schema definitions for data sources that appear in most queries.
- Build Derived Views for recurring multi-source joins: If Claude regularly needs cross-source data, pre-join it rather than letting Claude orchestrate multiple calls.
- Schedule Jobs for non-real-time data: Any query where hourly or daily freshness is acceptable should use caching.
- Package mature workflows as AI Skills: Once a workflow is stable and well-tested, convert it to a Skill to eliminate reasoning overhead.

Link to CData Connect AI documentation and the Connect AI product page as next steps.

Frequently Asked Questions

What causes high token usage when using Claude with enterprise data?

Example Answer: The primary drivers are large tool definition lists (schemas for many connected sources), multi-step tool call chains where Claude issues sequential queries, and verbose raw API responses. Architectural choices — how many tools Claude sees, how data is pre-joined, and whether results are cached — have a larger impact on token usage than prompt wording alone.

Does token optimization affect the quality of Claude's responses?

Example Answer: When done correctly, it improves response quality by reducing noise. Scoped tool lists and pre-joined Derived Views give Claude less irrelevant information to reason over, which leads to more focused and accurate outputs. The goal is eliminating context that doesn't contribute to the answer, not removing information Claude needs.

How do I measure token usage in my current Claude + Connect AI setup?

Example Answer: Use the Anthropic token counting API (POST /v1/messages/count_tokens) to measure input tokens before sending a request. Output tokens are reported in the usage field of

each API response. Log both fields per call to establish a baseline before applying Connect AI optimization features.

Which Connect AI feature provides the largest token reduction?

Example Answer: This depends on your query patterns. For teams with many connected data sources, Workspaces and Toolkits typically provide the largest gains by reducing the tool list footprint. For teams running frequent multi-source joins, Derived Views have the greatest impact. The benchmark in this article measures each feature in isolation — replace the placeholder values with results from your own environment for a precise answer.

Is token optimization only relevant for Claude, or does it apply to other AI models?

Example Answer: The architectural principles apply to any token-based LLM (OpenAI GPT models, Google Gemini, etc.). Connect AI's MCP server design means the same Workspaces, Toolkits, Derived Views, and AI Skills that reduce token usage with Claude will have equivalent effects with other MCP-compatible AI clients.

References & Links

Internal Link Candidates (sorted by relevance)

- <https://www.cdata.com/ai/> — CData Connect AI product page
- CData Derived Views documentation (link to internal KB or docs.cdata.com when published)
- CData Workspaces documentation (internal KB)
- CData AI Skills documentation (internal KB)
- <https://www.cdata.com/blog/> — CData Enterprise Data Integration Blog

External References

- [1] Anthropic. Token counting. <https://docs.anthropic.com/en/docs/build-with-claude/token-counting>
- [2] Anthropic. Pricing. <https://www.anthropic.com/pricing>
- [3] Anthropic. Tool use (function calling). <https://docs.anthropic.com/en/docs/build-with-claude/tool-use>
- [4] Model Context Protocol specification. <https://modelcontextprotocol.io/docs>
- [5] CData Connect AI. Managed MCP Platform for Enterprise AI. <https://www.cdata.com/ai/>